분산병렬컴퓨팅 MRTA 보고서

이현욱, 이상엽, 최원우

1. Introduction

Multi robot task scheduling Problem을 해결하기 위한 알고리즘을 개발한다. drone은 가장 시야 범위가 높기 때문에 오직 task의 위치 탐색과 UNKNOWN의 좌표를 탐색하는데 이용하였고, 나머지 Wheel과 Caterpillar는 발견된 task에 한해 task를 수행하는데 이용하였다. 따라서 알고리즘의 개발 과정은 크게 아래 2개의 subproblem으로 나눠진다.

a. Drone map search algorithm

b. Task assigning algorithms

보고서는 각각의 알고리즘의 아이디어와 구현 전략에 대해 설명한다. 프로젝트는 해당 개발 환경에서 수행되었다.

Language : c++17

Compiler : MSVC C++17 최적화 컴파일러 버전 19.40.33811

Build System: Msbuild 버전 17.10.4+10fbfbf2e(.NET Framework용) 17.10.4.21802

Os : x86용 Microsoft (R) Windows 10

2. Drone Map Search Algorithm

2.1 Idea

Drone은 map을 최대한 넓게 돌아다니며 UNKNOWN을 밝혀야 한다. 이를 위해 방문한 곳은 최대한 다시 방문하지 않아야 하지만, task는 지속적으로 map에 나타나기 때문에 어느 정도는 다시 방문해야 한다. 구현을 위해 개미군집 알고리즘(Ant Colony Optimization)의 아이디어를 차용하였다. 개미군집 알고리즘은 TSP(Travelling Salesman Problem) 문제를 확률적으로 해결하기 위한 알고리즘으로, 개미가 이동 경로에 pheromone을 남기며, 이 pheromone의 농도를 기반으로 다음 경로를 휴리스틱하게 결정한다. 이 아이디어를 바탕으로, 각 로봇이 pheromone을 지도 상에 남기도록 하였고, Drone의 탐색 방향은 이 pheromone의 수치를 기반으로 결정되도록 구현하였다. 이때 pheromone은 일정 비율로 decay 되도록 설정하였기 때문에, 과거에 방문한 지점을 일정 수준 다시 방문하게 되는 특성을 갖는다. Drone이 탐색하는 타겟은 pheromone이 가장 낮은 지점을 우선적으로 탐색하게 하여 하나의 알고리즘으로 초기 map 탐색(unknown을 줄이는 탐색)과 맵이 밝혀지고 나서의 반복 탐색을 구현하였다.

2.2 implementation

각 로봇은 시야 범위에 pheromone을 남기면서 지나다닌다. Pheromone map에 기록되는

pheromone 값은 매 틱마다 시야 범위 내에 정해진 값이 계속 누적되며 int 최대 값인 INF를 넘기지 않도록 설정했다. 그리고 남겨진 pheromone의 농도가 높을수록 최근에 방문한 좌표이므로 매 tick마다 농도가 가장 낮음 방향으로 결정하도록 하였다. 이때 시야 범위에 pheromone을 같은 농도로 남기게 된다면 그림 1에서 볼 수 있듯이 랜덤함수와 별 차이가 없게 되어버린다. 이유는 양쪽 방향에 side를 비교하여 농도가 가장 낮은 지점을 방문하게 되는데 농도가 같다면 이미 지나온 방향과 지나오지 않은 방향의 농도가 같기 때문이다. 따라서 각 로봇별 pheromone의 농도를 그림2. 처럼 설정하여 효율적인 탐색이 가능하도록 하였다.

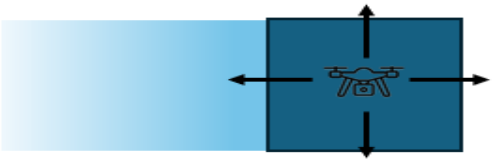

그림 1. Pheromone을 같은 농도

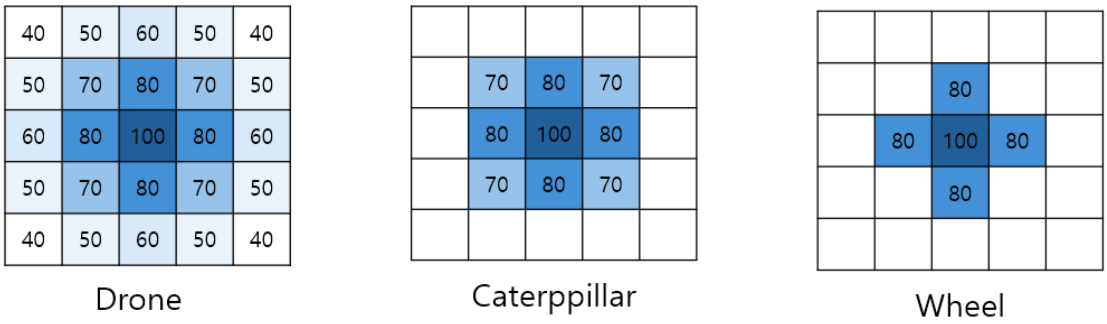

그림 2. 로봇별 Pheromone 농도

2.3 Improvement

2.1에서 구현한 drone map search algorithm으로 worst case분석을 진해하였다. 100회 실행 기준으로 상위 10% 평균 결과 Best case의 경우 모든 94.57%의 map이 밝혀 졌고, 평균 8.7개의 task가 발견되었다. 그러나 하위 10% 평균인 worst case의 경우 map의 44%도 탐색하지 못하였고, 평균 2.1개의 task를 발견하였다. 해당 경우에 대해서 cost map 과 unknown map을 출력하여 분석을 진행하였다.

그림 3에서 볼 수 있듯이 흰색 drone이 pheromone을 map에 미리 남겨 놓게 되면 빨간색 drone이 더 넓은 unknown이 있는 곳(보라색 화살표)으로 나아가지 못하고, 국소적인 범위만 탐색하기 때문에 왔던 곳을 다시 돌아가는 것을 확인할 수 있다. 이는 더 넓은 범위를 탐색하고자 하는 원래 알고리즘의 취지와 맞지 않기 때문에 해당 부분에 대한 수정을 진행하였다.

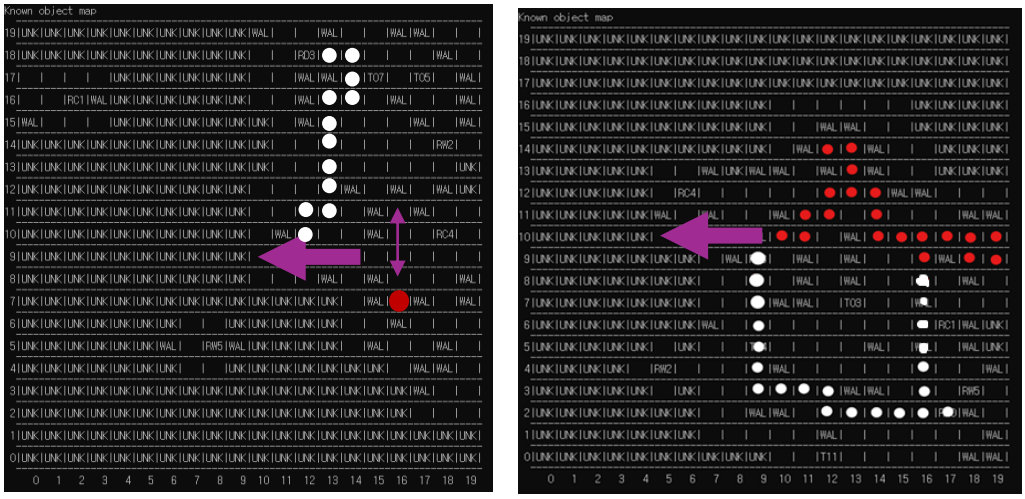

그림 3. Worst case 분석

해당 문제는 pheromone density map을 도입하여 해결하였다. Pheromone density map은 그림 4에서 보는 것처럼 5x5칸(또는 3x3칸)의 pheromone 밀도를 계산하여 실시간으로 기록한다. 이때 drone은 매 tick마다 양 4방향의 pheromone 농도를 비교하는 것이 아닌 phenomenon density가 가장 낮은 좌표로 target이 정해지고 그 target을 향해 방문하게 된다. 이때 pheromone density가 같다면 맨해튼 거리가 더 낮은 target으로 향하도록 한다.

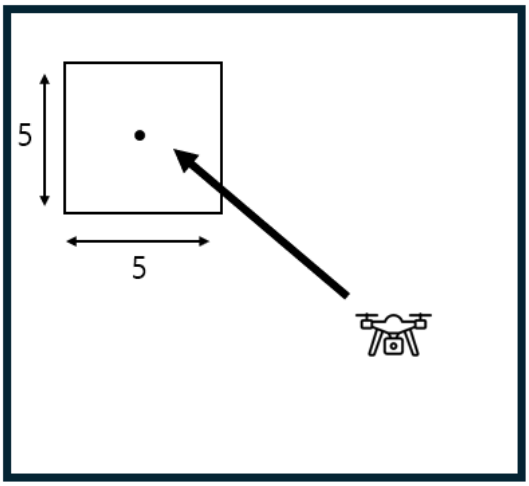

그림 4. Pheromone density map

target으로 drone이 도달하거나 target이 wall이므로 갈 수 없다는 것을 확인하면 새로운 target을 할당하고 알고리즘이 반복된다. 해당 algorithm은 Drone의 에너지가 다 떨어질 때까지 반복되고, 에너지가 다 소진되면 종료한다. Pheromone density map의 도입으로 drone이 map을 국소적으로 탐색하는 경우가 줄어들었다. 알고리즘을 수행할 때 자세한 파라미터는 벽의 경우 int 범위의 최댓값 0x3f3f3f3f을 할당하였고, UNKNOWN의 경우 0을 할당하여 최대한 벽 쪽으로는 향하지 않으면서 UNKNWON의 탐색을 촉진할 수 이 있도록 하였다.

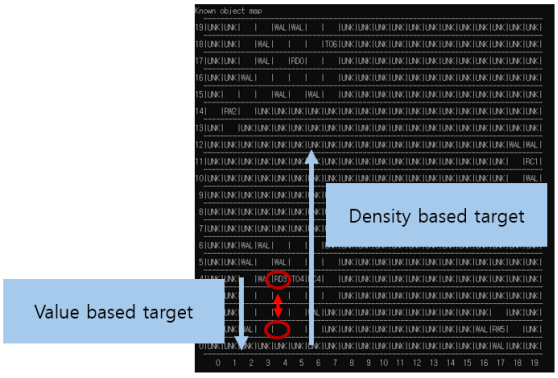

그림 5. Drone toggle문제

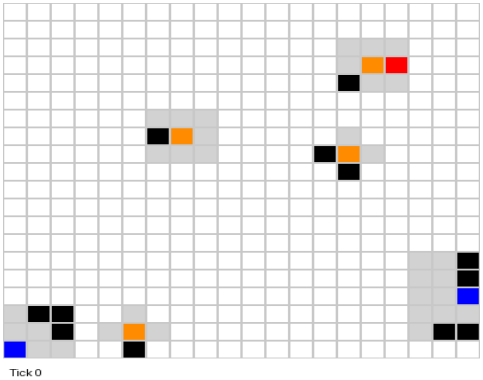

그림 6. Search 알고리즘 실행 결과

한번 설정된 target은 바뀌지 않기 때문에 만약 벽에 막혔을 경우에 대해서 예외 처리를 해줘야 한다. 벽에 막혔을 경우에는 value based로 양 4 방향의 국소적인 pheromone농도를 탐색하고 나아가게 된다. 하지만 이 방법은 치명적인 문제가 있는데 그림5에서 보는 것처럼 drone이 벽에 막혀 밑으로 진행하였다 다시 위로 target을 향해 일정하게 진행하게 된다. 따라서 드론이 toggle하면서 에너지를 다 써버리는 문제점이 있다 해당 문제를 해결하기 위해서 다익스트라 알고리즘을 이용하여 최소의 pheromone cost로 target으로 향하게 설정하였다. 최단 경로로 target으로 이동하게 끔 설정하면 toggle문제를 해결할 수 있는 이유는 벽으로 막혀서 진행하지 못하도 뒤돌아가게 되면 이미 지나왔으므로 pheromone을 남기게 되고 따라서 해당 지역은 피해서 돌아가게 된다. 또한 pheromone 다익스트라 방법은 최근에 이미 탐색된 구간, 즉 pheromone 농도가 높은 지역은 돌아서 가기 때문에 보다 효율적인 탐색이 가능하다.

또한, drone이 같거나 비슷한 위치의 target에 할당되었을 경우에는 비효율적인 탐색이 되어버리기 때문에 약간의 gap을 줘서 만약 같은 좌표가 할당이 되면 좀 멀리 떨어져 있는 곳으로 다시 target을 설정하도록 하였다. Gap 값은 map_size / 5로 설정하여 맵 사이즈에 따라 적절히 값이 달라질 수 있도록 했고, 20 x 20 map 기준으로는 가로 또는 세로의 타겟 좌표 차이가 4 초과하는 경우에 타겟 지점을 할당하도록 했다. 최종 알고리즘 실행결과 그림 6에서 보이는 것처럼 drone이 겹치지 않으면서 효율적으로 task들을 발견하는 것을 알 수 있다.

드론의 이동 바운더리를 맵 가장 바깥의 2칸은 이동하지 않는 것으로 제한하여 드론의 시야 범위 상 실제로 갈 필요가 없는 지점에 대한 비효율을 없앴고, 초기 드론의 스폰 위치가 바운더리의 바깥이 되는 경우에 바운더리 내부를 타겟으로 하는 이동을 전혀 하지 않기로 되어있기 때문에 움직이지 않는 문제를 해결하기 위해 예외적 처리로 BFS를 통해 바운더리 내부의 가장 가까운 좌표로 이동할 수 있도록 구현했다.

2.4 Search algorithm optimization

해당 search algorithm의 주요한 파라미터는 Edge pheromone과 pheromone decay rate이 있다. Edge pheromone은 초기에 drone의 탐색 방향을 설정하기 위해서 세팅 되어야하는 파라미터이다. 따라서 너무 높게 설정돼 있을 경우 map의 중심만 탐색하는 경향을 보이고 너무 낮게 설정되어

있으면 지나치게 가장자리만 탐색하는 경향을 보였다. 따라서 100개 평균을 기준으로 가장 task 할당이 잘 되는 개수를 찾아보았고, 결과 78이 적당하다는 결론을 얻을 수 있었다.

1x1은 map의 가장자리까지 탐색 가능 하지만 에너지 효율 안 좋고, 7x7은 테두리를 덜 가고, 중심파악에 효율적이라는 특징을 알 수 있었다. 따라서 적당한 사이즈, 3x3과 5x5에서 어느 것이 더 효과적일지 100회 평균을 기준으로 찾아보았고, 5x5가 가장 효과적임을 알 수 있었다.

	3 x 3	5 x 5
Average Activate task	8.5	11.3

약간의 예외 사항인 drone이 map 바깥에서 생성되는 경우에는 BFS함수를 이용해서 맨해튼 거리 기준으로 최단 거리

3. robot task assign algorithm

3.1 Idea

만약 task가 모두 주어졌을 경우 해당 문제는 최적해를 가지는 문제이지만 해당 프로젝트는 계속 새로운 task가 할당되기 때문에 최적해가 존재하니 않는 문제이다. minmin, suffrage 등 다양한 task 할당 알고리즘이 있는데 이것들은 모두 모든 task가 주어졌을 때 효과적인 알고리즘이기 때문에 최적화를 통해서 계속 task가 할당되는 상황에서의 알고리즘을 design해야 한다.

처음 8개가 할당될 때까지 기다렸다가 8개를 한번에 4대의 로봇에게 할당하고, 이후에 추가적으로 들어오는 task에 대해서 새롭게 scheduling하도록 한다. 처음 8개의 task할당에는 헝가리안 알고리즘을 이용하였다. 개수가 정해진 경우에 $O(n^3)$의 시간 복잡도로 확실하게 최적해를 찾을 수 있어서 해당 알고리즘을 선정하였다. 이후에 경우에 대해서는 suffrage를 이용하여 각 작업에 대해 1등과 2등 차이가 큰 작업을 우선적으로 배정하도록 알고리즘을 설계하였다.

3.2 implementation

헝가리안 알고리즘은 그림에서 보는 것처럼 task와 robot을 표에 할당하고 행의 최솟값과 열의 최솟값을 빼면서 최대한 많은 0을 만들고 line 2개가 0이되면 0이 아닌 것 중에서 최솟값을 뺀다. 이후 0이 하나만 있는 라인으로 시작하는 사람에게 장소를 할당 한다.

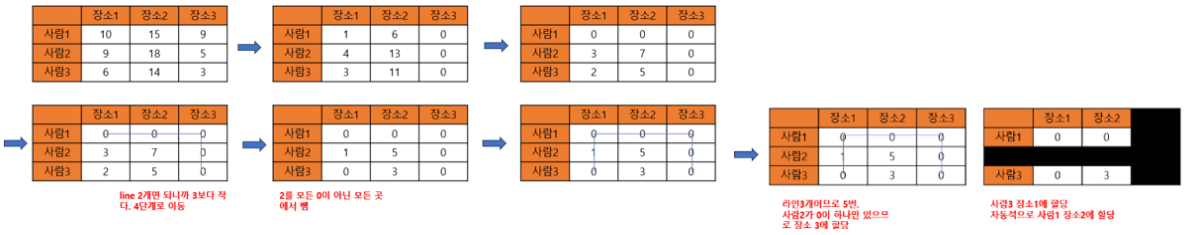

그림 7. 헝가리안 알고리즘

구현을 위해 처음에 정사각형 배열을 선언하였고, 이후에 들어오는 task에 대해 처리하기 위해 더미 행과 열을 추가하여 직사각형으로 확장하였다. 이후 새로운 task 발견 시에 로봇들끼리 모든

후보 작업에 대해 로봇 코스트 계산하였고, 이동 cost + task 처리 cost를 다익스트라 함수로 계산하여 최소 비용으로 task로 향하도록 하였다. 만약 여러 task가 동시에 할당되게 되면 코스트 1등과 2등의 차이가 큰 작업을 우선적으로 배정한다.

4. Result

본 프로젝트에서는 제한된 에너지 환경 하에서 작업(Task)을 최대한 많이 수행하도록 로봇들을 스케줄링하는 알고리즘을 설계하고, 그 성능을 시뮬레이션을 통해 평가하였다. 실험은 총 30회 반복 실행되었으며, 각 시뮬레이션에서 생성된 최대 작업 수는 16개였다.

평균적으로 로봇들이 **탐지에 성공한 작업 수는 9.46개**, **실제 수행 완료한 작업 수는 5.96개**로 나타났으며, 이는 전체 작업의 약 59.1%가 탐지되었고, 약 37.2%가 완료되었음을 의미한다. 이러한 결과는 로봇들이 에너지, 경로 거리, 작업 우선순위 등 다양한 요소를 고려하여 효율적으로 움직였음을 보여준다. 특히 탐지된 작업과 수행된 작업 사이의 간극은 에너지 제약 조건이 주요한 제약 요소로 작용했음을 시사한다.

한편, 가장 우수한 성능을 보인 Best Case에서는 13개의 작업이 성공적으로 완료되었으며, 일부 로봇은 모든 에너지를 소진한 상태에서도 마지막까지 작업을 수행하며 효율적인 자원 사용을 달성하였다. 이와 같은 결과는 제안한 스케줄링 방식이 일정 수준 이상의 작업 처리량을 안정적으로 보장할 수 있다는 가능성을 보여준다.

결과적으로 본 알고리즘은 제한된 자원과 불완전한 정보 환경에서도 비교적 높은 수준의 작업 처리율을 유지하며, 특히 탐색과 분배 간의 균형 조절이 시스템 전체 성능에 큰 영향을 준다는 점에서 실시간 로봇 스케줄링 문제에 있어 유의미한 해법으로 작용할 수 있음을 확인하였다.